



Annexe : Code Heltec emetteur Cooper

STN « 2025» _ « 2026 »

Développement
d'un système
embarqué pour
mesurer et
estimer la VO₂
max à partir des
tests de Léger et
Cooper.

Projet : STN21 – Système embarqué VO₂max

Auteur: Ben said Wassim

```

#include <HardwareSerial.h>    // Include hardware serial library for UART
communication
#include <TinyGPS++.h>        // Include TinyGPS++ library for parsing GPS
NMEA sentences
#include <RadioLib.h>         // Include RadioLib library for LoRa radio
communication

#define HELTEC_POWER_BUTTON    // Define power button for Heltec board
#include <Wire.h>              // Include Wire library for I2C communication
#include "SSD1306Wire.h"      // Include SSD1306 OLED display library
#include "pins_arduino.h"     // Include Arduino pin definitions

// ===== OLED =====
// Create OLED display object with I2C address 0x3C, using SDA and SCL pins
SSD1306Wire display(0x3C, SDA_OLED, SCL_OLED);

/**
 * Enables power supply for the OLED display
 * Vext is the external power control pin
 */
void VextON() {
    pinMode(Vext, OUTPUT);    // Configure pin as output
    digitalWrite(Vext, LOW);  // Set to LOW to enable (inverted logic)
}

/**
 * Disables power supply for the OLED display
 */
void VextOFF() {
    pinMode(Vext, OUTPUT);    // Configure pin as output
    digitalWrite(Vext, HIGH); // Set to HIGH to disable
}

/**
 * Resets the OLED display via its reset pin
 * Ensures the display starts up correctly
 */
void displayReset() {
    pinMode(RST_OLED, OUTPUT); // Configure reset pin as output
    digitalWrite(RST_OLED, HIGH); // Set high initially
    delay(1);
    digitalWrite(RST_OLED, LOW); // Pulse low for reset
    delay(1);
    digitalWrite(RST_OLED, HIGH); // Return to high state
    delay(1);
}

// ===== GPS =====
HardwareSerial gpsSerial(1);    // Use UART1 for GPS communication
const int RXPin = 45;          // GPS RX pin (receives data from GPS)
const int TXPin = 46;          // GPS TX pin (sends data to GPS)
const uint32_t GPSBaud = 9600; // GPS communication speed (standard
frequency)
TinyGPSPlus gps;               // Main GPS object for decoding NMEA
sentences

```

```

// ===== LoRa =====
#define LORA_FREQ 915.0 // LoRa frequency in MHz (915 MHz for US
region)
#define LORA_SS 8 // Chip Select (CS) pin for LoRa module
#define LORA_RST 12 // Reset pin for LoRa module
#define LORA_DI0 14 // Interrupt pin DI00 for LoRa module
Module loraModule(LORA_SS, LORA_DI0, LORA_RST); // Create module object with
pins
SX1262 radio(&loraModule); // Create radio object
using SX1262 chip

// ===== Distance & V02 =====
double prevLat = 0, prevLon = 0; // Last valid coordinates for distance
calculation
double distanceTotale = 0; // Total distance traveled in meters
bool firstFix = false; // Indicates if we already have a
reference position

// 12-minute chronometer
unsigned long startTime = 0; // Timestamp when test started
bool chronoStarted = false; // Indicates if chronometer is
running
const unsigned long DUREE_TEST = 12UL * 60UL * 1000UL; // 12 minutes in
milliseconds

// GPS status
bool fixObtained = false; // Indicates if a valid GPS fix has
been obtained

// GPS filtering
static double latSum = 0, lonSum = 0; // Sum accumulators for moving
average
static int count = 0; // Counter for moving average
const int N_MOY = 5; // Number of points for moving
average
const double MIN_DIST = 2.0; // Minimum distance to validate
movement (2 meters)

// ===== Functions =====
/**
 * Converts decimal degrees to radians
 * @param deg Angle in degrees
 * @return Angle in radians
 */
double toRad(double deg) {
    return deg * 3.141592653589793 / 180.0; // Multiply by  $\pi/180$  to convert
}

/**
 * Calculates distance between two GPS points using Haversine formula
 * @param lat1 Latitude of point 1
 * @param lon1 Longitude of point 1
 * @param lat2 Latitude of point 2
 * @param lon2 Longitude of point 2
 * @return Distance in meters
 */

```

```

double distanceGPS(double lat1, double lon1, double lat2, double lon2) {
    const double R = 6371000.0;           // Earth's radius in meters
    double dLat = toRad(lat2 - lat1);      // Difference in latitude in
radians
    double dLon = toRad(lon2 - lon1);      // Difference in longitude in
radians
    lat1 = toRad(lat1);                    // Convert first latitude to
radians
    lat2 = toRad(lat2);                    // Convert second latitude to
radians
    // Haversine formula:  $a = \sin^2(\Delta\text{lat}/2) + \cos(\text{lat1}) * \cos(\text{lat2}) * \sin^2(\Delta\text{lon}/2)$ 
    double a = sin(dLat/2)*sin(dLat/2) +
cos(lat1)*cos(lat2)*sin(dLon/2)*sin(dLon/2);
    double c = 2 * atan2(sqrt(a), sqrt(1-a)); // Central angle in radians
    return R * c;                          // Distance = radius × central
angle
}

// ===== Setup =====
void setup() {
    Serial.begin(115200);                  // Initialize serial communication at 115200
baud for debugging
    delay(500);                           // Small delay for stabilization

    // ---- OLED Initialization ----
    VextON();                             // Enable power supply for OLED display
    displayReset();                        // Reset OLED display hardware
    display.init();                         // Initialize OLED display
    display.flipScreenVertically();        // Set vertical orientation (based on
mounting)
    display.setFont(ArialMT_Plain_10);    // Set standard font size
    display.clear();                       // Clear display buffer
    display.drawString(0,0,"Init GPS + LoRa..."); // Display initial message
    display.display();                     // Update physical display

    // ---- GPS Initialization ----
    // Initialize GPS on UART1 with defined RX/TX pins
    gpsSerial.begin(GPSBaud, SERIAL_8N1, RXPIN, TXPIN);
    Serial.println("GPS initialisé");      // Print debug message
    display.drawString(0,12,"GPS OK");     // Confirm GPS OK on OLED
    display.display();

    // ---- LoRa Initialization ----
    // Manual reset of LoRa module
    pinMode(LORA_RST, OUTPUT);             // Configure reset pin as output
    digitalWrite(LORA_RST, LOW);           // Set LOW to trigger reset
    delay(10);
    digitalWrite(LORA_RST, HIGH);          // Return to HIGH to start module
    delay(10);

    // Initialize LoRa radio
    int state = radio.begin(LORA_FREQ);    // Start radio at defined frequency
    if (state != 0) {                      // If initialization failed
        display.drawString(0,24,"LoRa ERROR"); // Display error message
        display.display();
    }
}

```

```

    while(1); // Halt program execution
}
Serial.println("LoRa OK"); // Print success message
display.drawString(0,24,"LoRa OK"); // Confirm LoRa OK on OLED
display.display();
delay(800); // Delay for reading the message
}

// ===== Loop =====
void loop() {
    // Read all available GPS data from serial port
    while (gpsSerial.available()) {
        gps.encode(gpsSerial.read()); // Feed each byte to TinyGPS++ parser
    }

    // Check if GPS position is valid
    if (!gps.location.isValid()) {
        // If no signal, display searching message
        display.clear();
        display.drawString(0,0,"Recherche GPS..."); // "Searching GPS..."
        display.display();
        return; // Exit loop function
    }

    // First valid GPS fix obtained
    if (!fixObtained) {
        fixObtained = true; // Store that we have GPS fix
        display.clear();
        display.drawString(0,0,"GPS Fix OK !"); // Display fix confirmation
        display.display();
        delay(500); // Pause for display reading
        startTime = millis(); // Start chronometer
        chronoStarted = true; // Activate chronometer
    }

    // Retrieve current GPS data
    double lat = gps.location.lat(); // Latitude in decimal
    degrees
    double lon = gps.location.lng(); // Longitude in decimal
    degrees
    double alt = gps.altitude.isValid() ? gps.altitude.meters() : 0; // Altitude
    in meters
    int sat = gps.satellites.isValid() ? gps.satellites.value() : 0; // Number
    of satellites
    unsigned long t = millis()/1000; // Elapsed time in
    seconds

    // GPS filtering with moving average
    latSum += lat; // Add current latitude to sum
    lonSum += lon; // Add current longitude to sum
    count++; // Increment sample counter

    // If we have enough samples for moving average
    if (count >= N_MOY) {
        double avgLat = latSum / count; // Calculate average latitude
        double avgLon = lonSum / count; // Calculate average longitude
    }
}

```

```

    if (!firstFix) {
        prevLat = avgLat;
        prevLon = avgLon;
        firstFix = true;
    } else {
        // Calculate distance from last reference position
        double d = distanceGPS(prevLat, prevLon, avgLat, avgLon);
        // Verify distance is plausible (between 2m and 50m)
        if (d >= MIN_DIST && d < 50) {
            distanceTotale += d;
            prevLat = avgLat;
            prevLon = avgLon;
        }
    }
    // Reset accumulators for next averaging period
    latSum = lonSum = 0;
    count = 0;
}

// Calculate VO2max using Cooper test formula
// VO2max = (distance_meters - 504.9) / 44.73
double vo2 = (distanceTotale - 504.9) / 44.73;

// ---- OLED Display Update ----
display.clear();
display.drawString(0,0,"Sat: " + String(sat));
satellites // Number of
display.drawString(0,12,"Lat: " + String(lat,5));
decimal places // Latitude (5
display.drawString(0,24,"Lon: " + String(lon,5));
(5 decimal places) // Longitude
display.drawString(0,36,"Dist: " + String(distanceTotale,1)); // Distance (1
decimal)
display.drawString(0,48,"VO2: " + String(vo2,1)); // VO2max (1
decimal)
display.display(); // Update
physical display

// ---- Periodic LoRa Transmission ----
static unsigned long lastSend = 0; // Timestamp of last transmission
if (millis() - lastSend > 3000) { // Send every 3 seconds
    lastSend = millis(); // Update last send time

    // Build data packet string
    String packet =
        "LAT:" + String(lat,6) + // Latitude (6 decimal places)
        ",LON:" + String(lon,6) + // Longitude (6 decimal places)
        ",ALT:" + String(alt,1) + // Altitude (1 decimal)
        ",SAT:" + String(sat) + // Number of satellites
        ",DIST:" + String(distanceTotale,1) + // Total distance
        ",VO2:" + String(vo2,1) + // Calculated VO2max
        ",TIME:" + String(t); // Elapsed time in seconds

    radio.transmit(packet); // Transmit packet via LoRa
}

```

```
// ---- Test Completion ----
if (millis() - startTime >= DUREE_TEST) {    // If 12 minutes have elapsed
    display.clear();
    display.drawString(0,0,"FIN 12 MIN");      // Display test end message
    display.drawString(0,12,"Dist: " + String(distanceTotale)); // Final
distance
    display.drawString(0,24,"VO2: " + String(vo2));      // Final
VO2max
    display.display();
    while(1);      // Halt program execution
}
}
```